🌐 CEH v13 — Module 03: Scanning Networks

> **Personal lab notes & command reference** | Certified Ethical Hacker (C|EH) v13
> Documented for learning and revision purposes. No exam answers or proprietary content reproduced.

📌 What is Network Scanning?

Network scanning is **active reconnaissance** — you're no longer just watching from a distance. You're sending crafted packets to the target to map out its attack surface.

Unlike passive OSINT, scanning creates **direct contact** with target systems, and leaves traces. This is why it must only be performed with explicit authorization.

Three types of scanning:

- **Port Scanning** — What ports are open? What services are listening?
- **Network Scanning** — Which hosts are alive on the network?
- **Vulnerability Scanning** — Are those services known to be exploitable?

What you're trying to learn:

- Live hosts and their IP addresses
- Open ports and running services
- Service version numbers (for exploit matching)
- OS type and version (banner grabbing / fingerprinting)
- Network topology and security perimeter weaknesses

> Scanning is the bridge between **reconnaissance** and **exploitation**. Every open port is a potential door.

🗂️ Labs Index

#	Lab	Tool	Key Technique
1	Host Discovery	Nmap	ARP, ICMP, UDP, TCP ping scans
2	Port & Service Discovery	Nmap / Zenmap	TCP connect, SYN, Xmas, UDP, version scan
3	OS Discovery	Nmap NSE	Banner grabbing, SMB fingerprinting
4	Scan Beyond IDS/Firewall	Nmap	Fragmentation, decoys, MTU, source port

#	Lab	Tool	Key Technique
5	Network Scanning Tools	Metasploit	portscan modules, SMB version
6	AI-Assisted Scanning	ShellGPT	Natural language → automated scan pipelines

---

Lab 1: Host Discovery

🧠 Concept

Before you can scan ports or fingerprint services, you need to know **which hosts are actually alive**. Sending port scans to dead IPs wastes time and creates unnecessary noise.

Host discovery sends lightweight probes to determine whether a target is up. Different probe types are needed because:

- Firewalls may block ICMP but allow TCP
- Some hosts ignore pings but respond to ARP
- Network layer differences affect what gets through

> ARP requests cannot be spoofed or filtered within a LAN — making ARP ping the most reliable discovery method on local networks.

🖥️ Environment

Role	Machine	IP
Attacker	Parrot Security (Linux)	—
Target	Windows Server 2022	10.10.1.22
Sweep Range	Mixed hosts	10.10.1.10–23

⚙️ Commands & Explanations

```bash
nmap -sn -PR 10.10.1.22
```

> **One-liner:** Send an ARP request to the target IP — if an ARP reply comes back, the host is up. Most reliable discovery method on the same LAN segment since ARP cannot be filtered by host-based firewalls.

Flag	Meaning
`-sn`	**Disable port scan** — host discovery only, no port probing
`-PR`	**ARP ping scan** — send ARP request, wait for ARP reply

```bash
nmap -sn -PU 10.10.1.22
```

> **One-liner:** Send a UDP packet to an unlikely port — a responsive host either replies or sends an ICMP "port unreachable" error; either way, it proves the host is alive.

Flag	Meaning
`-PU`	**UDP ping scan** — sends to a high/unlikely port (default 40125)

```bash
nmap -sn -PE 10.10.1.22
```

> **One-liner:** Send an ICMP Echo Request (standard ping) to the target — if it replies with an Echo Reply, the host is up. Useful for testing whether ICMP passes through firewalls.

Flag	Meaning
`-PE`	**ICMP ECHO ping** — classic ping probe

```bash
nmap -sn -PE 10.10.1.10-23
```

> **One-liner:** Sweep an entire IP range with ICMP Echo Requests simultaneously — identifies all live hosts in a subnet in one pass. Equivalent to pinging every IP in the range at once.

> This is called an **ICMP ECHO ping sweep** — used to map out which IPs in a range are occupied.

```bash
nmap -sn -PP 10.10.1.22
```

> **One-liner:** Send an ICMP Timestamp Request to the target — an alternative to ECHO ping that can bypass firewalls specifically blocking ECHO requests, while still confirming the host is alive.

Flag	Meaning
`-PP`	**ICMP Timestamp ping** — queries current time from target host

---

📋 Full Host Discovery Flag Reference

Flag	Technique	How It Works
`-PR`	ARP Ping	ARP request → ARP reply. LAN only. Cannot be firewalled at host level.
`-PU`	UDP Ping	UDP to unlikely port → ICMP "port unreachable" or UDP reply
`-PE`	ICMP ECHO Ping	ICMP Echo Request → ICMP Echo Reply
`-PP`	ICMP Timestamp	ICMP Timestamp Request → Timestamp Reply
`-PM`	ICMP Address Mask	Alternative when ECHO is blocked by admin
`-PS`	TCP SYN Ping	SYN packet → ACK response = host is up
`-PA`	TCP ACK Ping	ACK packet → RST response = host is up
`-PO`	IP Protocol Ping	Multiple IP protocol probes — any response = alive

---

🔑 Key Takeaways

- `-sn` is critical — without it, Nmap runs a full port scan after discovery
- **ARP ping** is king on local networks — works even when the host blocks all ICMP
- Sweeping a range (`10-23`) maps the entire subnet in one shot — efficient recon
- Different probe types are needed because **every network has different filtering rules**
- No response doesn't always mean the host is down — it may just be blocking your probe type

Lab 2: Port and Service Discovery

🧠 Concept

Once live hosts are identified, you need to know **what doors are open** on each host. Ports are the communication endpoints — each open port = a service accepting connections = a potential attack vector.

Different scan types interact with TCP's three-way handshake differently:

```
Normal TCP Handshake:    SYN → SYN+ACK → ACK → [connection established]
SYN (Stealth) Scan:      SYN → SYN+ACK → RST  [connection aborted — never logged]
```

> The key insight: **a port's response (or silence) to different packets reveals whether it's open, closed, or filtered by a firewall.**

🖥️ Environment

Role	Machine	IP
Attacker	Windows 11 (Zenmap GUI)	—
Target	Windows Server 2022	10.10.1.22

⚙️ Commands & Explanations

```
nmap -sT -v 10.10.1.22
```

> **One-liner:** Perform a full TCP connect scan — completes the entire three-way handshake on every port to determine if it's open. Most accurate but also most detectable; leaves entries in the target's connection logs.

Flag	Meaning
`-sT`	**TCP connect scan** — full 3-way handshake (SYN → SYN+ACK → ACK → RST)
`-v`	**Verbose** — show all hosts and ports including closed ones

TCP Connect Scan flow:

```
Client          Target
  ─── SYN ──►
  ◄── SYN+ACK ──
  ─── ACK ──►   [open — connection established]
  ─── RST ──►   [immediately close it]
```

```
nmap -sS -v 10.10.1.22
```

> **One-liner:** Stealth scan — sends SYN packets but never completes the handshake, leaving connections half-open. Bypasses many logging mechanisms and firewall rules because no full session is established.

Flag	Meaning
`-sS`	**TCP SYN / Half-open / Stealth scan**

SYN Scan flow:

```
Client          Target
  ─── SYN ──►
  ◄── SYN+ACK ──  [port is open]
  ─── RST ──►  [abort — never completes — not logged by most apps]
```

```
nmap -sX -v 10.10.1.22
```

> **One-liner:** Xmas scan — sends packets with FIN, URG, and PUSH flags all set simultaneously ("lighting up like a Christmas tree"). Open ports ignore it; closed ports reply with RST. Results in "open|filtered" when a firewall is present.

Flag	Meaning
`-sX`	**Xmas scan** — FIN + URG + PUSH flags set

```
nmap -sM -v 10.10.1.22
```

> **One-liner:** Maimon scan — sends FIN/ACK probe; if the port is open or filtered, there's no response; if closed, the target sends RST. Named after the researcher Uriel Maimon who discovered it.

Flag	Meaning
`-sM`	**TCP Maimon scan** — FIN/ACK probe

```
nmap -sA -v 10.10.1.22
```

> **One-liner:** ACK flag probe scan — sends ACK packets to determine if ports are filtered (no response = stateful firewall present) or unfiltered (RST = port reachable). Used to **map firewall rules**, not find open ports.

Flag	Meaning
`-sA`	**ACK flag probe scan** — maps firewall rules

> ⚠️ This scan does **not** find open ports — it tells you whether a **stateful firewall** is in front of those ports.

```
nmap -sU -v 10.10.1.22
```

> **One-liner:** UDP scan — sends UDP packets to each port; no response means the port is open (UDP is connectionless); ICMP "port unreachable" means the port is closed. Slow but essential for finding UDP services.

Flag	Meaning
`-sU`	**UDP scan** — no handshake, relies on ICMP unreachable for closed ports

> ⏱️ UDP scans are slow — can take 20+ minutes. Run with `--top-ports 100` to limit scope.

```
nmap -sV 10.10.1.22
```

> **One-liner:** After finding open ports, probe each one to identify the exact software and version running — critical for matching ports to known CVEs and exploit code.

Flag	Meaning
`-sV`	**Service/version detection** — banner grabbing + probe matching

---

```
nmap -A 10.10.1.*
```

> **One-liner:** Aggressive all-in-one scan of an entire subnet — runs OS detection, version scanning, script scanning, and traceroute simultaneously. Loud and thorough; only use with permission.

Flag	Meaning
`-A`	**Aggressive scan** = `-O` + `-sV` + `-sC` + `--traceroute`
`10.10.1.*`	Wildcard = scan entire `/24` subnet

📋 Scan Type Decision Matrix

Scan Type	Flag	Logged by Target?	Works Through Firewall?	Purpose
TCP Connect	`-sT`	✅ Yes	✅ Yes	Reliable, full scan
SYN/Stealth	`-sS`	❌ Usually not	⚠️ Partial	Stealthy recon
Xmas	`-sX`	❌ No	❌ Blocked → open|filtered	Firewall detection
Maimon	`-sM`	❌ No	❌ Blocked → open|filtered	Legacy systems
ACK Probe	`-sA`	❌ No	N/A	Map firewall rules
UDP	`-sU`	❌ Usually not	⚠️ Partial	Find UDP services
Version	`-sV`	⚠️ Maybe	✅ Yes	CVE matching
Aggressive	`-A`	✅ Yes	⚠️ Partial	Full enumeration

🔑 Key Takeaways

- `-sT` is the most reliable but noisiest — avoid in real engagements unless necessary

- `-sS` requires **root/admin privileges** — it crafts raw packets
- Xmas and Maimon scans only work against **RFC-compliant TCP stacks** — modern Windows may not respond as expected
- `-sA` is a **firewall mapper**, not a port finder — easy to confuse
- Always follow up open ports with `-sV` to identify exploitable version numbers
- Zenmap's **Topology tab** provides a visual network map from scan results

---

Lab 3: OS Discovery

🧠 Concept

Knowing **which operating system** a target runs narrows down exploits dramatically. A Windows Server 2019 host has different vulnerabilities than an Ubuntu 22.04 server, even on the same open port.

OS discovery works by analyzing differences in TCP/IP stack implementations — different OSes respond slightly differently to the same crafted packets.

Two approaches:

- **Active fingerprinting** — send crafted packets and analyze responses
- **Passive fingerprinting** — observe traffic characteristics (TTL, window size) without sending probes

TTL & TCP Window Size Reference:

OS	Default TTL	TCP Window Size
Windows	128	65,535
Linux	64	5,840
Cisco IOS	255	4,128
macOS	64	65,535
FreeBSD	64	65,535

> TTL value decreases by 1 at each hop. If you see TTL=124, the host is likely Windows (128 - 4 hops = 124).

🖥️ Environment

Role	Machine	IP
Attacker	Parrot Security (Linux)	—
Target	Windows Server 2022	10.10.1.22

⚙️ Commands & Explanations

```bash
nmap -A 10.10.1.22
```

> **One-liner:** Run an aggressive scan that combines OS detection, version scanning, and NSE script scanning — returns OS name, computer name, NetBIOS name, domain, workgroup, and system time under "Host script results."

```bash
nmap -O 10.10.1.22
```

> **One-liner:** Dedicated OS fingerprinting scan — analyzes TCP/IP stack responses and TTL values to identify the operating system, version, and confidence percentage without version scanning noise.

Flag	Meaning
`-O`	**OS detection** — pure fingerprinting, no service probes

---

```bash
nmap --script smb-os-discovery.nse 10.10.1.22
```

> **One-liner:** Use the SMB protocol (ports 445/139) to directly query the target for OS, computer name, domain, workgroup, and current time — more reliable than TCP/IP fingerprinting for Windows hosts.

Part	Meaning
`--script`	Load and run an NSE (Nmap Scripting Engine) script
`smb-os-discovery.nse`	Queries SMB for OS and domain info — Windows-specific

Output includes:

```
Host script results:
  OS: Windows Server 2022 Standard
  Computer name: WIN-SERVER22
  NetBIOS computer name: WIN-SERVER22
  Domain: CEH.com
  System time: 2024-01-15T10:30:00
```

🔑 Key Takeaways

- `nmap -O` requires at least one **open and one closed port** to fingerprint accurately
- SMB-based OS discovery is more reliable on Windows than generic TCP fingerprinting
- TTL-based passive OS detection works **without sending any crafted packets** — completely silent
- OS confidence percentage in Nmap output (e.g., `93%`) indicates how certain the match is
- Combine `-O` with `-sV` for both OS and service version in one pass: `nmap -O -sV <target>`

Lab 4: Scan Beyond IDS and Firewall

🧠 Concept

IDS (Intrusion Detection Systems) and firewalls are designed to block or alert on scanning activity. However, they have limitations that can be exploited through **evasion techniques** — making your scan traffic look like legitimate traffic or making it too small/ambiguous to trigger rules.

Why evasion matters: A scan that gets blocked or alerted on tells the target you're there. Evasion = gathering data without triggering defenses.

Evasion techniques covered:

Technique	Core Idea
Packet Fragmentation	Break packets into tiny pieces — IDS may skip fragmented traffic
Source Port Manipulation	Pretend traffic comes from port 80/443 — firewall allows known ports
MTU Manipulation	Set small Maximum Transmission Unit — same effect as fragmentation
IP Address Decoy	Mix real scan IP with fake IPs — IDS can't tell which is real

---

🖥️ Environment

Role	Machine	IP
Attacker	Parrot Security (Linux)	—
Target (Firewall ON)	Windows 11	10.10.1.11
Monitoring	Wireshark on Windows 11	—

⚙️ Commands & Explanations

```bash
nmap -f 10.10.1.11
```

> **One-liner:** Fragment IP packets into 8-byte chunks before sending — many IDS systems skip or fail to reassemble tiny fragments, allowing the scan to pass through undetected while still returning port results.

Flag	Meaning
`-f`	**Fragment packets** — splits probe into 8-byte fragments

Wireshark filter to verify: `ip.fragment`

```bash
nmap -g 80 10.10.1.11
```

> **One-liner:** Set the source port of all scan packets to port 80 (HTTP) — firewalls configured to allow inbound traffic from trusted ports like 80 or 443 may pass this traffic without inspection.

Flag	Meaning
`-g 80`	**Source port manipulation** — disguise scan as HTTP traffic
`--source-port`	Alias for `-g`

> Real-world use: `-g 53` mimics DNS traffic, `-g 443` mimics HTTPS — match to whatever the target firewall trusts.

```bash
nmap --mtu 8 10.10.1.11
```

> **One-liner:** Manually set the Maximum Transmission Unit to 8 bytes — forces all packets to be fragmented to this size. More controlled than `-f` and can bypass IDS rules tuned to detect specific fragment sizes.

Flag	Meaning
`--mtu 8`	**Custom MTU** — must be a multiple of 8; smaller = more fragments

Wireshark observation: All captured packets have maximum length = 8 bytes.

```bash
nmap -D RND:10 10.10.1.11
```

> **One-liner:** Generate 10 random decoy IP addresses and mix them with the real scan traffic — the IDS/firewall sees 11 different source IPs and cannot determine which one is the actual attacker.

Flag	Meaning
`-D`	**Decoy scan** — add fake source IPs to obscure real attacker
`RND:10`	Generate **10 random** non-reserved IPs as decoys
`-D 192.168.1.5,ME,RND:3`	Manual decoys with `ME` = your real IP mixed in

Wireshark observation: Multiple different source IPs appear for the same scan session.

📋 Full IDS/Firewall Evasion Reference

Technique	Nmap Flag	What It Does
Packet fragmentation	`-f`	Splits packets into 8-byte pieces
Custom MTU	`--mtu <size>`	Control exact fragment size (multiples of 8)
Source port	`-g <port>`	Fake trusted source port (80, 443, 53)
IP decoy	`-D RND:<n>`	Mix n random fake IPs with real scan
IP decoy (manual)	`-D ip1,ip2,ME`	Use specific fake IPs
Spoof source IP	`-S <IP>`	Fake your source IP entirely (blind scan)
Random host order	`--randomize-hosts`	Scan IPs in random order, not sequential
Slow scan	`-T0` or `-T1`	Throttle scan speed below IDS detection thresholds
Bad checksums	`--badsum`	Send packets with invalid checksums
Proxy chains	via proxychains	Route scan through proxy chain

🔑 Key Takeaways

- Fragmentation (`-f`) bypasses **stateless** packet filters but not **stateful** firewalls that reassemble before inspecting
- Source port manipulation works only on poorly configured firewalls that trust source ports
- IP decoys don't hide you completely — the traffic still originates from your actual IP, just mixed in
- `-T0` (Paranoid) timing makes scans extremely slow but nearly invisible to IDS rate-based detection
- Always pair evasion techniques: `nmap -f -g 80 -D RND:5 <target>` combines multiple vectors
- These techniques are detectable by **next-gen firewalls** and **behavioral IDS** — evasion is an arms race

---

Lab 5: Network Scanning using Metasploit

🧠 Concept

Metasploit isn't just an exploitation framework — it has a rich library of **auxiliary scanning modules** that complement Nmap. The advantage of using Metasploit for scanning is that discovered hosts and services automatically populate the **Metasploit database**, creating a structured target profile that feeds directly into exploitation modules.

Metasploit scanner modules used:

Module	Purpose
`auxiliary/scanner/portscan/syn`	SYN scan across IP ranges
`auxiliary/scanner/portscan/tcp`	Full TCP connect scan
`auxiliary/scanner/smb/smb_version`	SMB version → OS fingerprinting

🖥️ Environment

Role	Machine	IP Range
Attacker	Parrot Security (Linux)	—
Target Subnet	Mixed hosts	10.10.1.0/24
Focused Target	Windows Server 2022	10.10.1.22

⚙️ Commands & Explanations

Launch Metasploit:

```bash
msfconsole
```

> **One-liner:** Start the Metasploit Framework console — provides an interactive environment for running exploit and auxiliary modules against discovered targets.

Run Nmap from inside MSF:

```
msf > nmap -Pn -sS -A -oX Test 10.10.1.0/24
```

> **One-liner:** Execute a Nmap aggressive SYN scan from within the Metasploit console, saving results to an XML file — the XML output can be imported directly into MSF's database for structured target tracking.

Flag	Meaning
`-Pn`	Skip host discovery — treat all hosts as alive
`-sS`	SYN/stealth scan
`-A`	Aggressive: OS + version + scripts + traceroute
`-oX Test`	Save output as XML file named "Test"

Find port scanning modules:

```
msf > search portscan
```

> **One-liner:** Search Metasploit's module library for all port scanning auxiliaries — returns a list of available scanners organized by technique (SYN, TCP, ACK, etc.).

SYN scan via Metasploit:

```
msf > use auxiliary/scanner/portscan/syn
msf > set INTERFACE eth0
msf > set PORTS 80
msf > set RHOSTS 10.10.1.5-23
msf > set THREADS 50
msf > run
```

> **One-liner:** Load MSF's SYN port scanner, configure it to check port 80 across a range of IPs using 50 concurrent threads — identifies which hosts in the range have port 80 open.

Option	Meaning
`INTERFACE`	Network interface to use for the scan
`PORTS`	Port(s) or range to scan (`22-25,80,443`)
`RHOSTS`	Target IP, range (`10.10.1.5-23`), or CIDR
`THREADS`	Concurrent scan threads — higher = faster but noisier

TCP scan via Metasploit:

```
msf > use auxiliary/scanner/portscan/tcp
msf > set RHOSTS 10.10.1.22
msf > run
```

> **One-liner:** Run a full TCP connect scan against a single target using Metasploit's TCP scanner — enumerates all open TCP ports and stores results in the MSF workspace database.

OS discovery via SMB version:

```
msf > use auxiliary/scanner/smb/smb_version
msf > set RHOSTS 10.10.1.5-23
msf > set THREADS 11
msf > run
```

> **One-liner:** Query the SMB service on all targets in the range to identify their Windows version and Samba version — SMB version maps directly to OS version for accurate fingerprinting.

🔑 Key Takeaways

- Metasploit stores scan results in its **workspace database** — discovered hosts/ports persist and are available to exploitation modules automatically
- `THREADS` controls speed vs. stealth — high thread counts are faster but generate detectable traffic bursts
- `smb_version` is highly reliable for OS fingerprinting Windows hosts — SMB version maps 1:1 to Windows build
- `search` is your friend inside MSF — always search before assuming a module doesn't exist

- Nmap's `-oX` output can be imported into MSF with `db_import Test.xml` to skip redundant scanning

---

Lab 6: Network Scanning using AI (ShellGPT)

🧠 Concept

ShellGPT is an AI-powered CLI tool that translates **natural language prompts into shell commands**. For network scanning, this means describing what you want to achieve in plain English — and getting executable commands in return.

This represents a shift in pentesting workflow: instead of memorizing exact flag syntax, you describe intent and the AI constructs the command. You still need to **understand the output** — the AI doesn't replace expertise, it accelerates execution.

ShellGPT chat mode (`--chat scan`) maintains context across prompts, allowing multi-step workflows where each command builds on the previous one.

⚙️ Key Prompts & What They Generate

```bash
sgpt --chat scan --shell "Use hping3 to perform ICMP scanning on 10.10.1.11, stop af
```

> **One-liner:** Ask ShellGPT to construct an hping3 ICMP scan command — demonstrates using AI to generate tool-specific syntax without memorizing hping3's flags.

```bash
sgpt --chat scan --shell "Run a hping3 ACK scan on port 80 of target IP 10.10.1.11"
```

> **One-liner:** Translate "ACK scan on port 80" into the correct hping3 command — ShellGPT resolves the right flags and parameters automatically.

```bash
sgpt --chat scan --shell "Scan 10.10.1.0/24 for active hosts and place only IP addre
```

> **One-liner:** Describe a subnet sweep with file output — ShellGPT generates the appropriate nmap/arp-scan command with output redirection. Results saved to scan1.txt for use in

> subsequent scans.

```bash
sgpt --chat scan --shell "Run a fast but comprehensive nmap scan against scan1.txt w
```

> **One-liner:** Chain scans — use the IP list from the previous step as input for a detailed Nmap scan. ShellGPT maintains the context of `scan1.txt` from the prior prompt in the same `--chat` session.

```bash
sgpt --chat scan --shell "Use nmap to perform ICMP ECHO ping sweep on 10.10.1.0/24"
```

> **One-liner:** Natural language → `nmap -sn -PE 10.10.1.0/24`. ShellGPT resolves the correct Nmap flag for ICMP ECHO ping sweep without requiring knowledge of `-PE`.

```bash
sgpt --chat scan --shell "Perform stealth scan on 10.10.1.11 and display results"
```

> **One-liner:** Translates "stealth scan" → `nmap -sS` automatically — useful for quickly executing known techniques without referencing documentation.

```bash
sgpt --chat scan --shell "Use Nmap to scan open ports, MAC details, services on open
```

> **One-liner:** Ask for a composite scan covering ports, MAC addresses, and service versions — ShellGPT constructs the multi-flag Nmap command needed.

```bash
sgpt --chat scan --shell "Use TTL value to identify the OS on 10.10.1.11, display TT
```

> **One-liner:** TTL-based passive OS fingerprinting via natural language — ShellGPT generates a ping or nmap command that extracts TTL and maps it to an OS.

```bash
sgpt --chat scan --shell "To evade IDS/Firewall, use IP address decoy technique to s
```

> **One-liner:** Translates "IP decoy evasion" → `nmap -D RND:n` — demonstrates using AI to apply specific evasion techniques by describing the goal rather than the syntax.

```bash
sgpt --chat scancode --code "Create a python script to run comprehensive Nmap scan o
```

> **One-liner:** Generate a complete Python automation script that chains Nmap and Nikto (web vulnerability scanner) — demonstrates AI-assisted scripting for automated pentest workflows.

🔑 Key Takeaways

- `--chat <name>` maintains conversation context — use the same name across prompts to build multi-step workflows
- `--shell` generates shell commands; `--code` generates full scripts
- Always type `E` to **Execute** or `D` to **Describe** (explain what the command does) before running
- ShellGPT doesn't replace security knowledge — you still need to understand what the generated commands do and interpret results
- AI-generated commands can vary between runs — always review before executing
- The `--code` flag is powerful for building automation: scan → filter → exploit chains in a single script

---

🛠️ Complete Tool & Flag Reference

Nmap Host Discovery

Flag	Technique	Response = Host Up
`-PR`	ARP Ping	ARP Reply
`-PU`	UDP Ping	UDP reply or ICMP unreachable
`-PE`	ICMP ECHO	ICMP Echo Reply
`-PP`	ICMP Timestamp	Timestamp Reply
`-PM`	ICMP Address Mask	Mask Reply
`-PS`	TCP SYN Ping	SYN+ACK or RST
`-PA`	TCP ACK Ping	RST
`-PO`	IP Protocol Ping	Any response

Nmap Port Scan Types

Flag	Scan Type	Handshake State	Privileges
`-sT`	TCP Connect	Full (SYN→SYN+ACK→ACK→RST)	User
`-sS`	SYN/Stealth	Half-open (SYN→SYN+ACK→RST)	Root
`-sU`	UDP	None (connectionless)	Root
`-sX`	Xmas	FIN+URG+PUSH sent	Root
`-sM`	Maimon	FIN+ACK sent	Root
`-sA`	ACK Probe	ACK sent (maps firewall)	Root
`-sI`	IDLE/IPID	Spoofed source	Root
`-sY`	SCTP INIT	INIT chunk	Root
`-sZ`	SCTP COOKIE ECHO	COOKIE ECHO chunk	Root

Nmap Output & Enhancement Flags

Flag	Effect
`-v`	Verbose output
`-A`	Aggressive: OS + version + scripts + traceroute
`-O`	OS detection only
`-sV`	Service version detection
`--script <name>`	Run specific NSE script
`-oX <file>`	XML output (importable to Metasploit)
`-oN <file>`	Normal text output
`-oG <file>`	Grepable output
`-T<0-5>`	Timing: 0=slowest/stealthy, 5=fastest/loud
`-p <ports>`	Specify ports (e.g., `-p 22,80,443` or `-p 1-1000`)
`-Pn`	Skip host discovery, scan anyway
`--top-ports <n>`	Scan the n most common ports

Metasploit Scanning Modules

Module	Use
`auxiliary/scanner/portscan/syn`	SYN scan across ranges
`auxiliary/scanner/portscan/tcp`	Full TCP connect scan
`auxiliary/scanner/portscan/ack`	ACK firewall mapping
`auxiliary/scanner/smb/smb_version`	OS via SMB version
`auxiliary/scanner/ftp/ftp_version`	FTP service version

🔐 Defensive Takeaways (Blue Team Perspective)

Attack Technique	Detection Method	Mitigation
ARP Ping sweep	ARP flood detection	Dynamic ARP inspection, rate limiting
ICMP ECHO sweep	ICMP rate monitoring	Rate-limit ICMP at perimeter
TCP SYN scan	Half-open connection detection	SYN cookies, stateful firewall
Xmas / Maimon scan	Unusual TCP flag combinations	Drop packets with invalid flag combos
Packet fragmentation	Fragment reassembly + inspection	Next-gen firewall with deep inspection
Source port spoofing	Don't trust source ports alone	Stateful firewall, not just ACL-based
IP decoy scan	Correlate TTL inconsistencies	Behavioral analysis, not IP-only blocking
SNMP-style enumeration	Monitor for sweep patterns	Honeypots, IDS rate rules
OS fingerprinting	Normalize TCP responses	TCP/IP stack fingerprint normalization

---

📎 Notes

- **Lab environment:** CEH v13 iLabs (EC-Council official)
- **Target IPs used:** 10.10.1.11 (Win11), 10.10.1.19 (WinServer 2019), 10.10.1.22 (WinServer 2022)
- **Attacker machines:** Parrot Security OS (Linux), Windows 11
- All techniques performed in a **controlled, authorized lab environment**

Module 03 complete. See Module 04 — Enumeration for the next phase.